



BME513 - Inteligencia Artificial en Salud

Residual Networks

Alejandro Veloz

Learning goals

- Explain why simply adding layers can make optimization worse.
- Describe residual connections as additive updates.
- Connect skip paths to gradient flow and smoother loss surfaces.
- Explain why BatchNorm is paired with residual blocks.
- Recognize ResNet, DenseNet, U-Net, and hourglass design patterns.

From CNNs to very deep CNNs

AlexNet: about 8 learned layers.

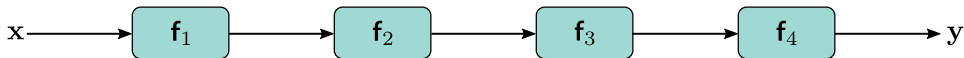
VGG: about 19 learned layers.

Natural question:

If 19 layers are better than 8, why not 50, 100, or 200?

More depth gives more representational capacity, but not necessarily easier optimization.

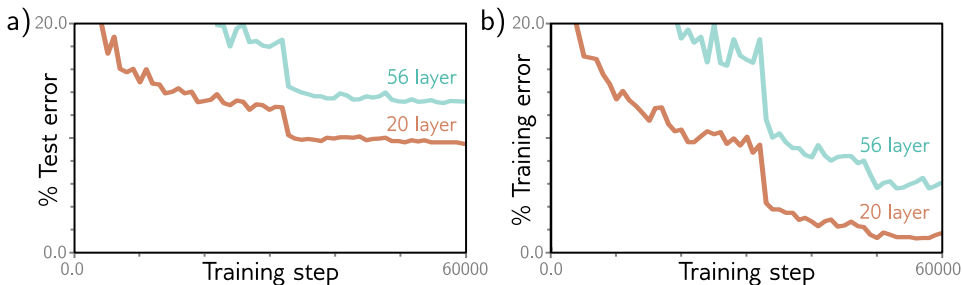
Sequential processing



In a plain network, each layer must pass all useful information to the next layer.

$$y = f_4(f_3(f_2(f_1(x))))$$

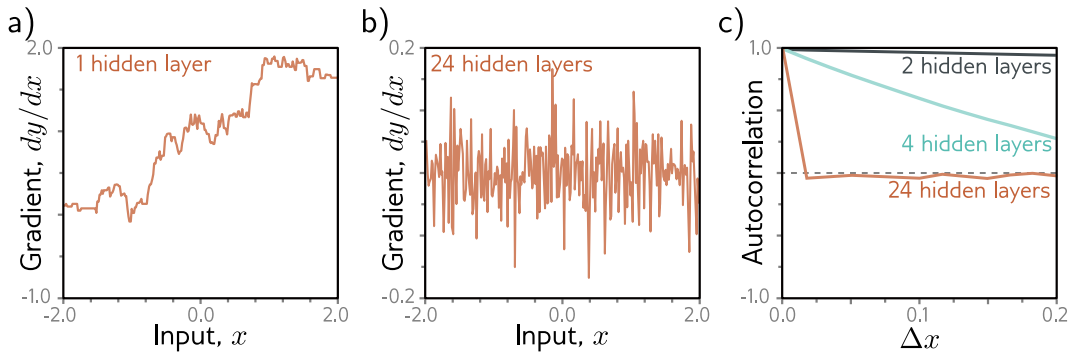
The degradation problem



A deeper plain CNN can have worse training error than a shallower one.

That points to an optimization problem, not just overfitting.

Shattered gradients phenomenon



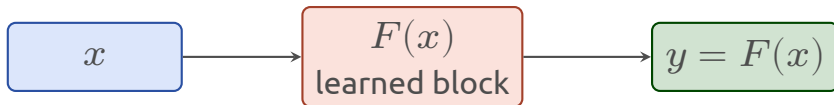
Gradient descent assumes local smoothness.

In very deep plain networks, nearby inputs or nearby parameter settings can produce unrelated gradients.

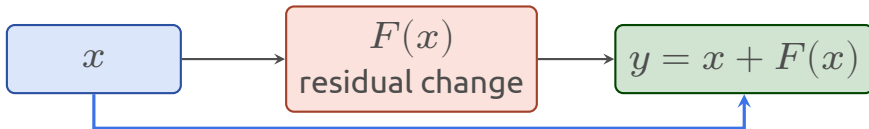
Residual connections

Plain vs residual block

Plain block: learn the full transformation



Residual block: learn an additive update



Instead of learning a full transformation, the block learns a correction to the current representation.

Residual network equations

Plain layer:

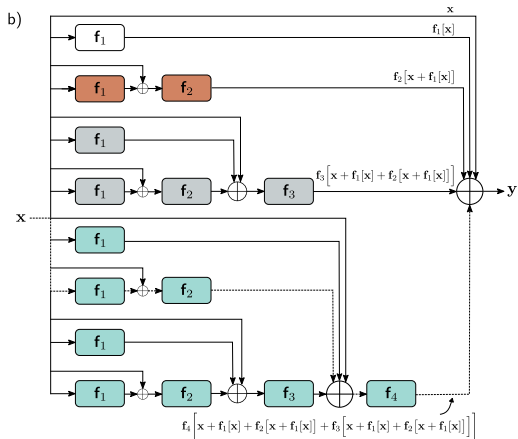
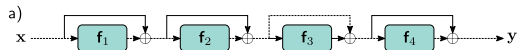
$$h_k = f_k(h_{k-1}).$$

Residual layer:

$$h_k = h_{k-1} + f_k(h_{k-1}).$$

The shortcut path carries an identity contribution forward.

Unraveling the network



A residual net can be viewed as many paths of different lengths through the computational graph.

Why skip paths help gradients

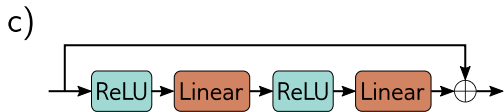
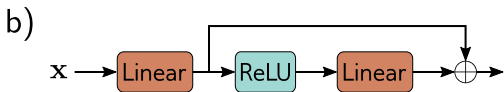
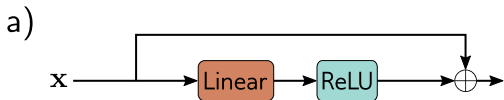
For a plain chain, early layers receive products of many derivatives.

$$\frac{\partial y}{\partial f_1} \approx \frac{\partial f_2}{\partial f_1} \frac{\partial f_3}{\partial f_2} \dots$$

With residual paths, the derivative includes shorter routes, including identity-like terms.

Training does not rely only on the longest path.

Order of operations matters

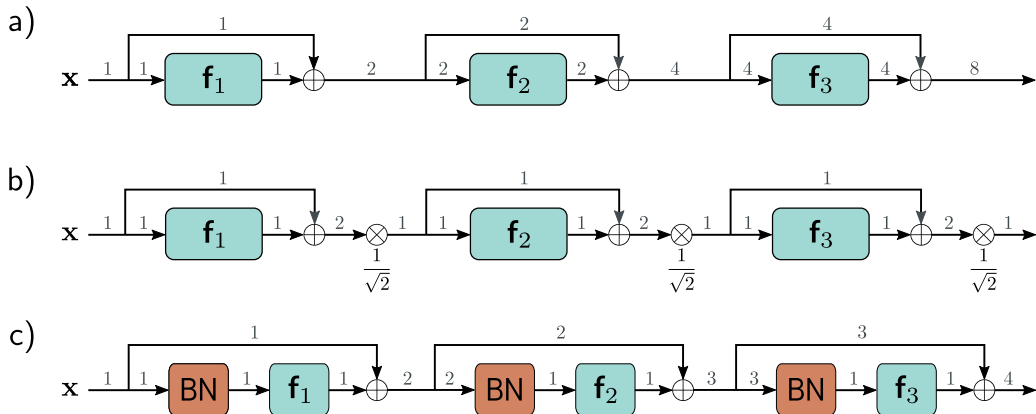


If a residual branch ends with ReLU, it can only add nonnegative changes.

Modern residual blocks often place normalization and activation before convolution.

Batch normalization

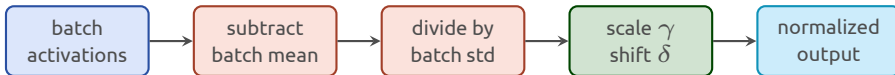
The variance problem



Residual connections prevent information loss, but adding two branches can make activation variance grow rapidly.

BatchNorm operation

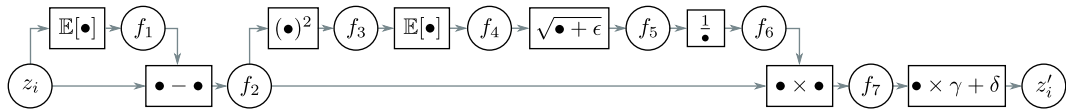
BatchNorm recenters, rescales, then learns the useful scale



For batch $\mathcal{B}$:

$$h_i \leftarrow \gamma \frac{h_i - m_h}{s_h + \epsilon} + \delta.$$

BatchNorm in residual blocks



BatchNorm helps residual nets by:

- Stabilizing forward activation scales.
- Reducing gradient shattering.
- Allowing larger learning rates.
- Injecting batch-statistics noise as regularization.

Train mode vs eval mode

During training:

m_h, s_h are estimated from the mini-batch.

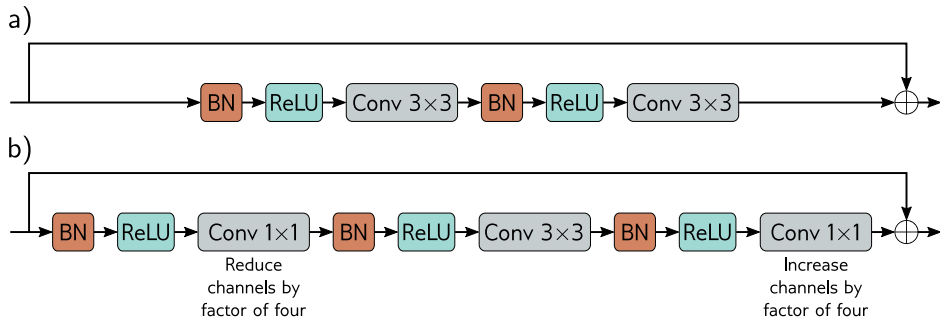
During inference:

m_h, s_h are fixed running estimates from training.

```
model.train()  # BatchNorm uses batch statistics  
model.eval()   # BatchNorm uses running statistics
```

Architectures

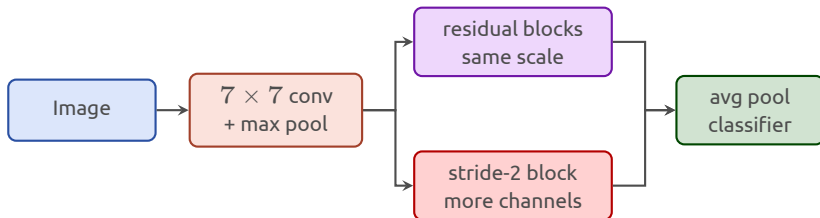
ResNet blocks



Bottleneck blocks use 1×1 convolutions to reduce and then restore channel dimension around a 3×3 convolution.

ResNet design pattern

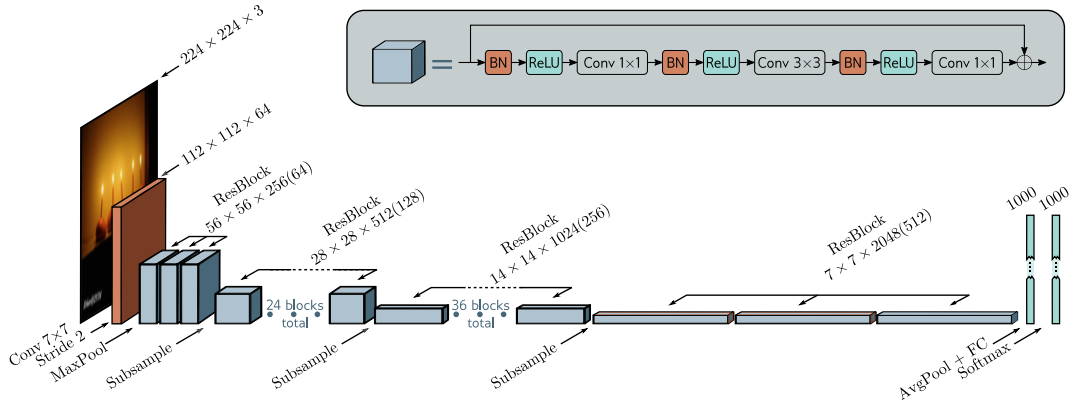
ResNet pattern: reduce spatial size, increase channels.



Depth comes mostly from repeated residual blocks.

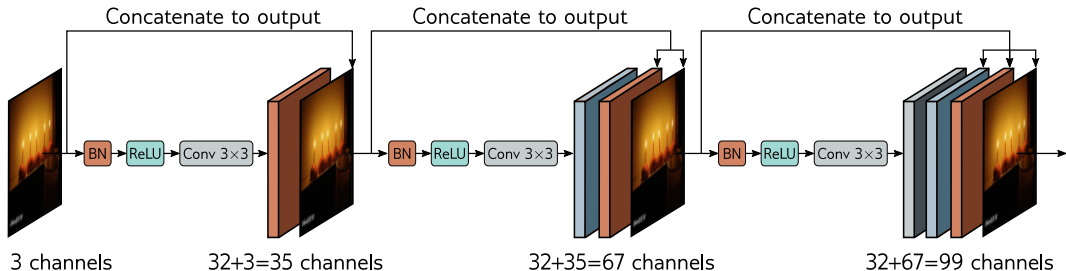
Like other CNNs, ResNets usually decrease spatial resolution while increasing channel depth.

ResNet-200



Very deep ResNets stack many bottleneck residual blocks between downsampling stages.

DenseNet

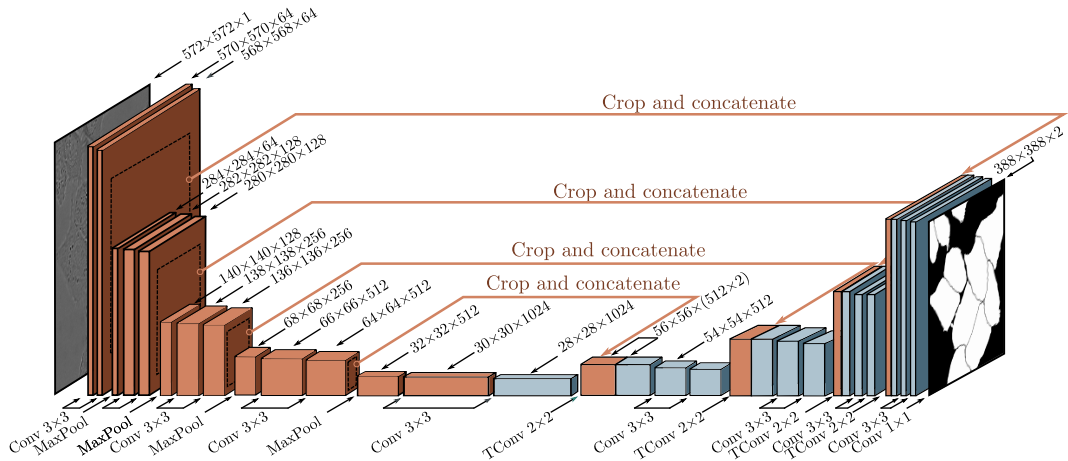


DenseNet concatenates earlier representations:

$$h_k = [h_0, h_1, \dots, h_{k-1}, f_k(\cdot)].$$

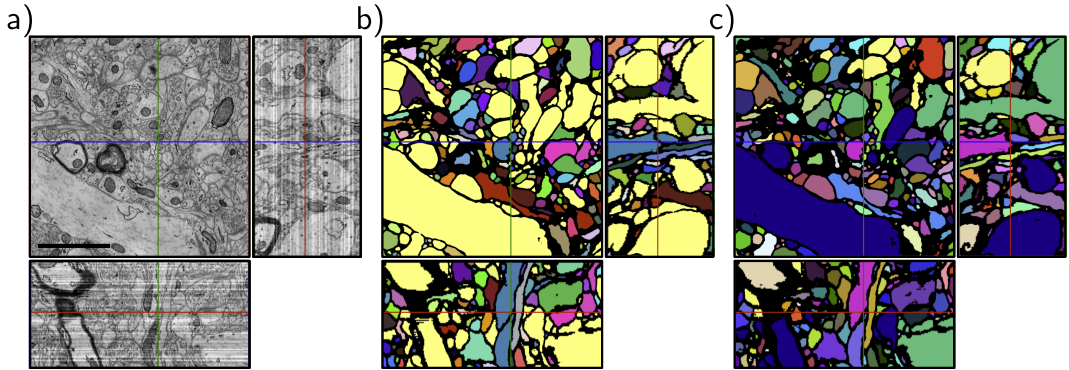
This encourages feature reuse but grows channels quickly.

U-Net



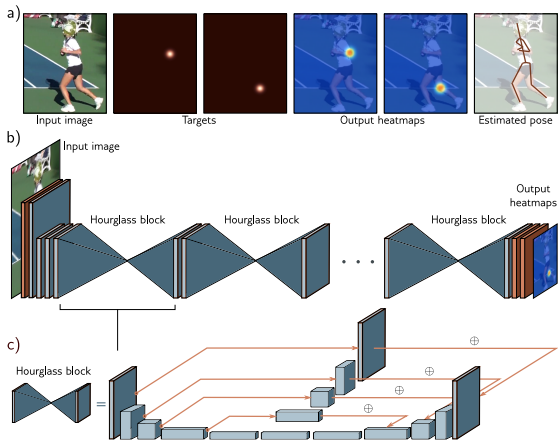
Encoder-decoder networks use skip connections to recover high-resolution spatial detail.

U-Net results



U-Nets are especially common in biomedical image segmentation.

Stacked hourglass networks



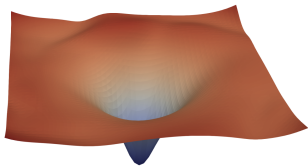
Hourglass models repeatedly move between local detail and global context.

Why residual nets work

Smoother loss surfaces

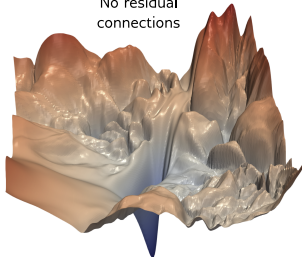
a)

Residual
connections



b)

No residual
connections



Residual connections and BatchNorm tend to make the optimization landscape smoother and more predictable.

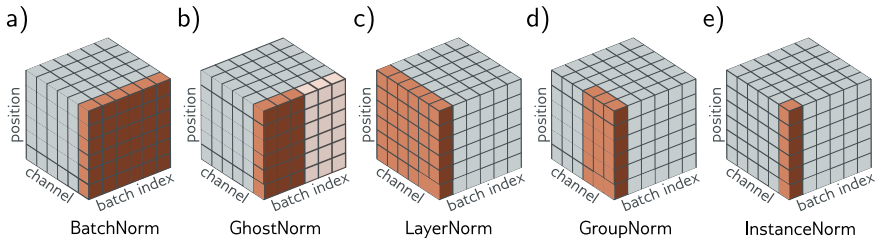
Depth is not the whole story

Residual networks made 100+ layer CNNs trainable.

But performance is not explained by depth alone:

- wider residual networks can outperform deeper narrow ones;
- gradients often travel mostly through shorter paths;
- residual blocks may act like an ensemble of shallower subnetworks.

Normalization variants



BatchNorm uses batch statistics; LayerNorm, GroupNorm, and InstanceNorm normalize over different axes.

Practical residual block in PyTorch

```
class BasicBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.block = nn.Sequential(
            nn.BatchNorm2d(channels),
            nn.ReLU(),
            nn.Conv2d(channels, channels, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(channels),
            nn.ReLU(),
            nn.Conv2d(channels, channels, kernel_size=3, padding=1, bias=False),
        )

    def forward(self, x):
        return x + self.block(x)
```

Summary

- Plain deep CNNs can become harder to optimize as depth increases.
- A residual block learns $F(x)$ and returns $x + F(x)$.
- Skip connections create short paths for information and gradients.
- BatchNorm stabilizes activation scales and helps smooth optimization.
- ResNet sums, DenseNet concatenates, U-Net skips across encoder and decoder.
- Residual connections are a general design principle, not just one architecture.